Grounded Literature Review Generation with Multi-Agent LLMs and Citation-Relation Learning

Introduction to Artificial Intelligence (CSS_2203)
<SEMESTER> SEMESTER MINI PROJECT REPORT
Submitted
to

MANIPAL ACADEMY OF HIGHER EDUCATION

School of Computer Engineering
SoCE – AI, <Section>

Submitted by

Siddharth Ray
Reg. No. 235890416
<STUDENT NAME>
<reg no.>
<STUDENT NAME>
<reg no.>

Under the guidance of

Dr. Simi V.R.
Assistant Professor- Senior Scale
School of Computer Engineering
Manipal Institute of Technology, Bengaluru

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

October 2026

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

ABSTRACT

Large language models write fluent literature reviews that cite papers they have not read. This project builds and evaluates a CrewAI-based research-paper summarisation and literature-review assistant in which every sentence of the output is traceable to a quoted passage of a specific paper and is checked by a natural-language-inference Verifier before it survives. Six agents — Librarian, Extractor, Verifier, Synthesist/Adjudicator, Gap Analyst and Writer — are orchestrated as a CrewAI Flow with a human approval checkpoint after corpus collection.

Training and evaluation data are reverse-engineered from real papers: a paper's related-work section is a human-written mini literature review, so mining those sections together with the abstracts of the papers they cite yields both a supervision signal and a natural benchmark. From 200 arXiv cs.CL and cs.IR submissions the pipeline produced 15,704 citation contexts and 80 usable related-work benchmark examples.

The full S1–S4 comparison had not been run when this report was built; Chapter 4 states exactly which systems were evaluated. All reported numbers are read from the result files the code wrote, and every figure is generated from those files by a single script.

Table of Contents

Table of Contents

			Page No
Abstract			ii
List of Tables			iii
List of Figures			iv
Chapter 1		**INTRODUCTION**	1
	1.1	Overview of the project work	1
	1.2	Motivation and background of the study	1
	1.3	Problem statement and relevance	2
	1.4	Objectives	2
Chapter 2		**BACKGROUND THEORY / LITERATURE REVIEW**	5
	2.1	Summary of existing studies / Comparative analysis of various approaches	8
	2.2	Key findings, trends, and research gaps	8
	2.4	Conclusion	9
Chapter 3		**METHODOLOGY**	10
	3.1	Methodology with diagrams	10
	3.2	System requirements	16
Chapter 4		**ANALYSIS AND DISCUSSION**	19
Chapter 5		**CONCLUSION AND FUTURE SCOPE**	25
	5.1	Conclusion	25
	5.2	Suggestions for future directions	25
References			28

LIST OF TABLES

Table No	Table Title	Page No
Table 1	Comparative analysis of approaches to grounded review generation	8
Table 2	System requirements and the exact versions used for the runs reported in Chapter 4	16
Table 3	Stage 1 mining yield at pilot scale	20
Table 4	Dataset statistics for the mined arXiv benchmark and the external Multi-XScience benchmark	20

LIST OF FIGURES

Page numbers below are Word fields. They were filled in when this document was rendered; if you edit the document, select all (Ctrl+A, or Cmd+A on macOS) and press F9 to refresh them.

Figure No	Figure Title	Page No
Figure 1	System architecture	10
Figure 2	Data-mining pipeline	11
Figure 3	The ten Flow steps, the human approval gate and the single revision loop	14
Figure 4	Mining yield: papers surviving each step, as counts and as a percentage of the submissions listed	20
Figure 5	Distribution of related-work length, number of cited papers and abstract coverage across the mined benchmark examples	20

CHAPTER 1
INTRODUCTION

1.1 Overview of the project work

This project builds a literature-review assistant from a team of large-language-model agents, and then measures whether the team is worth its cost. Given a topic or a few seed papers, the system collects a corpus, extracts a structured record from each paper in which every claim quotes the passage it came from, checks each claim against that passage with a natural-language-inference model, builds a consensus and contradiction map over the reported results, identifies candidate research gaps, and finally writes a related-work section in which each sentence carries the identifiers of the claims it rests on.

The pipeline is orchestrated as a CrewAI Flow of ten steps with a human approval checkpoint after corpus collection, and six agents with deliberately narrow roles: the Extractor cannot search, the Writer cannot verify, and only the Verifier may call the entailment tool. Two components are fine-tuned rather than prompted. The first is a seven-class citation-relation classifier, trained from SciBERT on citation contexts mined out of real papers' related-work sections. The second is a small instruction-tuned model distilled from the hosted extractor, trained only on extractions whose every claim passed the Verifier, so the Verifier acts as the filter on its own training data.

The same supervision idea supplies the benchmark. A published paper's related-work section is a literature review that a domain expert wrote for a known set of cited papers. Hiding the section and supplying only those cited papers' abstracts gives a task with a human reference, which is what makes the evaluation in Chapter 4 possible without any new human annotation effort at evaluation time.

1.2 Motivation and background of the study

Abstractive summarisation models are fluent and are often wrong in ways that fluency hides. Maynez et al. [1] found that a large share of summaries from strong sequence-to-sequence models contain content unsupported by the source document, and that ROUGE barely distinguishes those summaries from faithful ones. The survey of hallucination in natural language generation [2] documents the same failure across generation tasks. For a literature review the consequence is specific and serious: a plausible sentence attached to a real citation is a fabricated claim about somebody's work, and it is harder to catch than an invented reference, because the reference checks out.

The response in the summarisation literature has been to evaluate entailment rather than overlap. Question-answering-based consistency checks [3], classifier-based checks [4] and sentence-level natural-language-inference checks [5] all score a summary against its source instead of against a reference text. This project adopts that stance at the level of the individual sentence, and goes one step further by requiring the system to say exactly which

passage each sentence is supposed to follow from, so the check is well defined rather than approximate.

Multi-agent LLM frameworks have meanwhile made it cheap to decompose a task across specialised roles with tools and a shared state [6], [7], and self-critique loops [8], [9] have been shown to improve outputs on several tasks. What is largely missing is an honest accounting of whether that machinery pays for itself on a task with a human reference. The evaluation in this report is designed to answer that question for one task, with the same model and settings across every system, and to report the answer whichever way it comes out.

1.3 Problem statement and relevance

The problem addressed is: given a research topic and the papers relevant to it, produce a literature review in which every sentence is grounded in an identifiable passage of an identifiable paper, together with a map of where those papers agree and disagree and a ranked set of gaps, and do so under free-tier API limits and on a single free-tier GPU.

Three sub-problems follow. First, grounding needs supervision that does not exist as a labelled dataset: what relation a citing sentence asserts about a cited paper has to be learned, and the only abundant source of that signal is the related-work sections of published papers. Second, verification has to be mechanical rather than a second opinion from the same model, because a model asked to check its own output agrees with itself. Third, the whole thing has to be measured against something a human wrote, or the claim of groundedness is untestable.

The relevance is practical. The number of papers in any active area now exceeds what a student can read before writing a review, and the tools that promise to help are precisely the tools that hallucinate. A system that is slower but whose every sentence can be checked against a quoted passage is more useful than a faster one whose output must be re-read against the sources anyway.

1.4 Objectives

The objectives are numbered and measurable; the metric named in each is the one reported in Chapter 4.

1. Mine related-work sections and citation contexts from arXiv LaTeX sources, and report the yield at every step of the pipeline as a percentage of papers listed.
2. Define a seven-class citation-relation taxonomy with a written labelling guide, label contexts with an LLM, and quantify how far those labels can be trusted by measuring accuracy and Cohen's κ against an independently hand-labelled gold set.
3. Fine-tune SciBERT on those labels and beat a majority-class, a TF-IDF + logistic-regression and a zero-shot LLM baseline on macro-F1 over the gold set.

4. Extract structured records in which every claim quotes its evidence verbatim, and verify each claim with a quote check and an NLI entailment check whose thresholds are fitted on a hand-checked dev set rather than chosen by hand.

5. Build a consensus and contradiction map and a method-family × dataset coverage matrix, and have agents argue both sides of each flagged contradiction and each candidate gap using only the evidence.

6. Distil the hosted extractor into a small local model using only Verifier-approved extractions, and compare the two on JSON validity, claim faithfulness, field agreement, latency and cost.

7. Evaluate four systems and one ablation on a frozen, time-split benchmark with ROUGE, BERTScore, citation coverage, citation correctness and NLI faithfulness, reporting bootstrap 95% confidence intervals and the token and latency cost of each.

CHAPTER 2
BACKGROUND THEORY / LITERATURE REVIEW

This chapter surveys the five bodies of work this project draws on: multi-document and scientific summarisation, citation intent and function classification, faithfulness and hallucination evaluation, LLM agents and multi-agent frameworks, and LLM-assisted literature-review tools. Every work discussed here was checked against OpenAlex, Semantic Scholar or the arXiv API before being cited; the verification record is in results/references_verified.json.

Multi-document and scientific summarisation

Neural abstractive summarisation of single documents matured with pointer-generator architectures that could copy out-of-vocabulary tokens from the source [10] and then with pre-trained denoising sequence-to-sequence models, of which BART [11] and PEGASUS [12] remain the standard comparison points. Scientific articles broke the assumption that a document fits in the encoder: Cohan et al. [13] introduced a discourse-aware attention model together with the arXiv and PubMed long-document datasets, and the XSum corpus [14] pushed in the opposite direction, towards one-sentence summaries that cannot be produced extractively.

Related-work generation is the sub-task this project evaluates, and it is older than the current generation of models. Abu-Ra'ed et al. [15] treated it as scientific document abstracting over the cited papers, and Hoang and Kan's successors, notably Wang et al. [16], introduced joint context-driven attention over the citing paper's own content and the cited abstracts. Luu et al. [17] framed the problem as explaining the relationship between two documents rather than summarising one of them, which is the framing closest to the relation classifier used here.

Multi-XScience [18] is the dataset that made the task a benchmark: each example pairs a paper's related-work paragraph with the abstracts of the papers it cites, which is exactly the input-output contract adopted in Chapter 3. It is used in this project as a secondary benchmark alongside the mined arXiv split, so results can be compared against a published dataset rather than only against our own mining. The corpus-level infrastructure both rely on comes from S2ORC [19], which released parsed full text and resolved citation links at a scale that made citation-context mining routine. More recently, Zhang et al. [20] found that instruction-tuned LLMs match or beat fine-tuned summarisers in human judgement while reference-based metrics disagree, which is a direct warning about reading ROUGE alone, and one the evaluation design in this report takes seriously.

Citation intent and function classification

Not every citation does the same work, and the differences are what a review has to express. Early work in the area concentrated on citation sentiment and on recovering the citation's context window [21], [22]. The two datasets that standardised the task are ACL-ARC, whose six citation-frame labels Jurgens et al. [23] used to trace how a field changes, and SciCite [24], which traded label granularity for scale with three classes — background, method and result — and showed that auxiliary structural tasks improve classification. SciBERT [25] supplied the scientific-text encoder that both datasets are now usually attacked with, and is the starting point for the classifier in this project.

The taxonomy used here is finer than SciCite's and differently cut from ACL-ARC's, for a reason that matters downstream: this project needs to distinguish neutral comparison from asserted disagreement, because a contrasts edge is evidence of a contradiction while a compares-against edge is not. SciCite's result class and ACL-ARC's CompareOrContrast class both merge the two. Section 3.1 documents the mapping used to reuse those corpora as additional training signal, and the limits of that mapping; because of those limits, mapped rows are never used as evaluation gold.

Faithfulness and hallucination evaluation

The finding that motivates this project is that fluent summaries are frequently unfaithful and that reference-overlap metrics do not notice [1]. Three families of automatic check emerged in response. FactCC [4] trains a classifier on synthetically corrupted summaries. QAGS [3] generates questions from the summary and compares the answers obtained from the summary and from the source. SummaC [5] revisits sentence-level natural-language inference and shows that NLI models work well for inconsistency detection once they are applied at sentence granularity rather than to whole documents, which is the design this project's Verifier follows.

Evaluating the evaluators required human annotation: SummEval [26] re-annotated model outputs across several dimensions and showed how weakly the standard metrics correlate with human judgement, and the FRANK-style fine-grained factuality annotations of Goyal and Durrett [27] showed that error types differ systematically between model families. BERTScore [28] is included in this project's metric set as a semantic-similarity complement to ROUGE, not as a faithfulness measure, because it compares a generation against a reference rather than against a source.

The claim-verification literature supplies the other half of the picture. FEVER [29] established the retrieve-then-entail formulation at scale, SciFact [30] moved it into the scientific domain where claims are hedged and evidence is technical, and DeBERTa [31] provides the disentangled-attention encoder whose MNLI/FEVER/ANLI-trained checkpoints are used as the entailment model in Chapter 3. The survey of hallucination [2]

ties the strands together and is the reference for the terminology used in this report: a claim is unsupported when the evidence does not entail it, which is weaker and more checkable than calling it false.

LLM agents and multi-agent frameworks

Prompting a single model to reason in steps [32] was the first sign that decomposition helps, and tool use [33] made it possible for a model to act outside its own context. Multi-agent arrangements followed: CAMEL [6] showed that role-played agents conversing can carry a task further than a single prompt, and generative agents [7] demonstrated persistent memory and planning across long horizons. Iterative self-correction was formalised as Self-Refine [8] and, with an explicit verbal critic, as Reflexion [9].

CrewAI, the framework used here, belongs to this family: it provides typed agents with roles, goals and tools, tasks with structured outputs, and Flows that wire steps together with explicit control flow and state. Two of its features are load-bearing in this project. Structured task output validated against a Pydantic model is what lets the Extractor's output be checked mechanically rather than parsed from prose. Flows with named steps are what let the human approval checkpoint be a real gate and let a run resume after a disconnection, which matters when the whole pipeline must fit inside free-tier limits.

The critical observation from this literature, for present purposes, is that self-critique uses the same model that produced the text. A model's own judgement of its own output is correlated with the error that produced it. The Verifier in this project is therefore not an agent asked to be sceptical but a separate discriminative model applied to a well-defined pair of strings, and the agent whose role is called Verifier may do nothing except call it.

LLM-assisted literature review

Retrieval-augmented generation [34] with dense retrieval [35] is the standard way to ground a generator in documents, and it is the backbone of the applied systems. PaperQA [36] is a retrieval-augmented agent for scientific question answering that reports reduced hallucination relative to a plain LLM by forcing answers through retrieved passages. STORM [37] attacks the longer-form problem: it researches a topic by simulating multi-perspective question asking, builds an outline, and writes a long article with citations. Sušnjak et al. [38] fine-tune domain-specific models for research synthesis and report on the practical trade-offs of doing so.

Two things separate the system built here from those. The first is sentence-level accountability: STORM and PaperQA cite at the level of a passage or a paragraph, whereas this system records, for every sentence, the claim identifiers it used, and then checks that

sentence against exactly those claims' evidence. The second is that two components are learned rather than prompted. The citation-relation classifier is trained on data mined from real related-work sections, and the extractor is distilled with QLoRA [39] from the hosted model onto a small open base model [40], using the Verifier to filter the distillation set.

2.1 Summary of existing studies / works / Comparative analysis of various approaches

Table 1: Comparative analysis of approaches to grounded review generation. The grounding unit is what a generated statement can be traced back to; verification is what, if anything, checks that trace.

Approach	Representative work	Grounding unit	Verification	Typed relations	Learned component
Pre-trained seq2seq summariser	BART [11], PEGASUS [12]	none (whole source)	none	no	full fine-tune
Related-work generator	Wang et al. [16], Multi-XScience [18]	cited abstract	none	no	task-specific model
Citation intent classifier	SciCite [24], ACL-ARC [23]	citation sentence	n/a	3 or 6 classes	SciBERT fine-tune
NLI faithfulness checker	SummaC [5], QAGS [3]	summary sentence	entailment / QA	no	NLI model (off the shelf)
Retrieval-augmented agent	PaperQA [36], STORM [37]	retrieved passage	retrieval only	no	none (prompted)
Self-critique loop	Self-Refine [8], Reflexion [9]	whole output	same model critiques itself	no	none (prompted)
This project	Chapters 3–4	quoted passage per claim; claim ids per sentence	separate NLI model, thresholds fitted on a dev set	7 classes, fine-tuned	relation classifier + distilled extractor

2.2 Key findings, trends, and research gaps

Four trends run through the work above. Evaluation has shifted from reference overlap towards source entailment, because overlap metrics were shown not to track faithfulness. Grounding has become finer, from document-level retrieval towards passage- and sentence-level attribution. Citation classification has been standardised but at a granularity chosen for annotation cost rather than for downstream use. And orchestration has moved from single prompts to agent teams, with self-critique as the usual quality mechanism.

Three gaps follow, and they are what this project addresses. First, the faithfulness literature evaluates summaries against a source document, while the review-generation literature generates from many sources and evaluates against a reference text; almost nothing evaluates a multi-source review against the sources it cites, sentence by sentence. Second, citation-relation labels are used to study fields rather than to drive synthesis: no system known to us uses a contrasts prediction as evidence that two papers disagree and then asks an agent to adjudicate. Third, the cost of multi-agent pipelines is rarely reported next to their quality, so it is not possible to tell from the literature whether the extra calls are worth making.

A fourth observation is methodological rather than a gap in the literature. Several of the agent papers above report improvements without a time-split evaluation, on tasks whose

references may be inside the evaluated model's training data. Section 3.1 and docs/leakage.md therefore document the submission windows used here and why the test window begins after the knowledge cutoff of every model evaluated.

2.4 Conclusion

The components this project needs all exist and are individually well studied: pre-trained summarisers, citation-intent classifiers, NLI-based consistency checks, agent frameworks and retrieval-augmented generation. What has not been assembled is a pipeline in which the citation relation is learned from real related-work sections, every generated sentence names the evidence it rests on, a separate discriminative model adjudicates that evidence with calibrated thresholds, and the resulting system is measured against human-written sections on a time-split benchmark with its cost reported alongside its quality. Chapter 3 describes that assembly and Chapter 4 reports what it achieved, including where it did not beat the single-call baseline.

CHAPTER 3
METHODOLOGY

3.1 Methodology with diagrams

The system is one Python package with two execution paths over it: command-line scripts for local runs and six Colab notebooks for the full-scale runs. The notebooks install the package and call the same functions, so there is no second implementation that can drift. Figure 1 shows the components and how data moves between them.

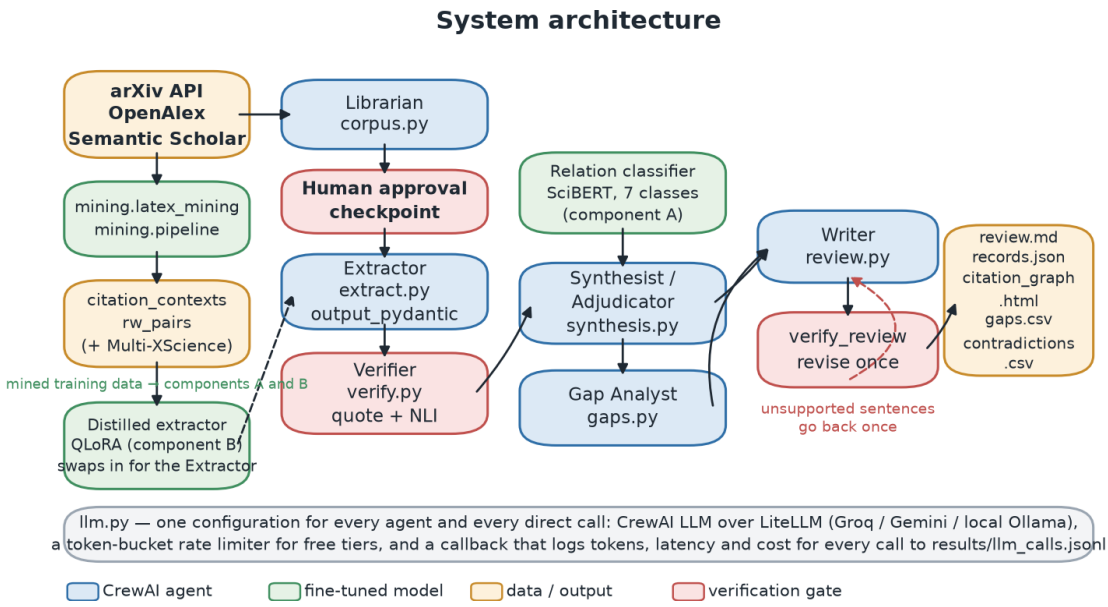


Figure 1: System architecture. Blue boxes are CrewAI agents, green boxes are fine-tuned models, red boxes are verification gates, and orange boxes are data or outputs. One LLM configuration serves every agent and logs the cost of every call.

3.1.1 Data mining and the supervision signal

The supervision for this project is reverse-engineered from published papers. A related-work section is a literature review that a domain expert wrote for a known set of cited papers, so mining those sections yields two things at once: citation contexts labelled implicitly by the sentence around them, and (section, cited abstracts) pairs that form a benchmark with a human reference. Figure 2 shows the pipeline.

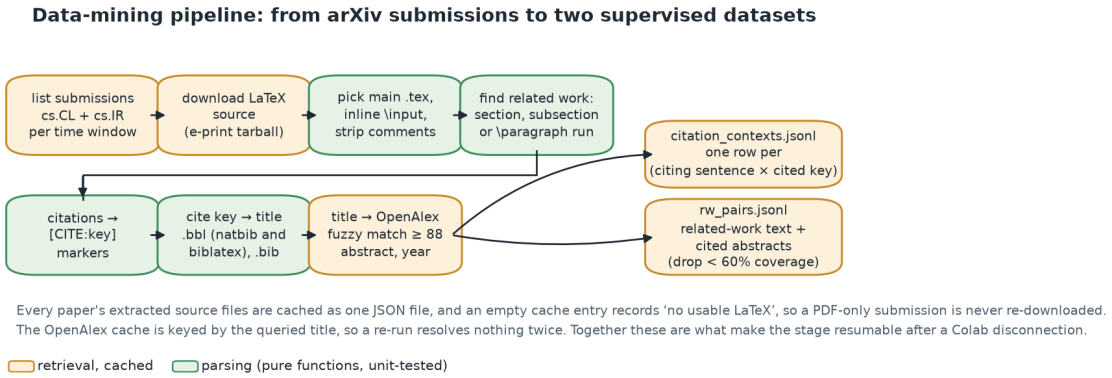


Figure 2: Data-mining pipeline. Each paper's extracted LaTeX is cached as one JSON file and an empty cache entry records a PDF-only submission, which is what makes the stage resumable.

The parser handles the cases a naive implementation fails on, each of which was found in the pilot and is covered by a unit test: several files carrying \documentclass (a supplement alongside the paper), related work that is a \subsection of a Background section or exists only as a run of \paragraph blocks, bibliographies in biblatex's \entry/\field form as well as the \bibitem/\newblock families, a thebibliography block inlined in the main file, and the full set of ACL, IEEE and ACM citation macros. A heading alone is not accepted as evidence of a related-work section: an explicit “Related Work” heading must contain at least one citation and a looser heading such as “Background” at least two, otherwise a technical primer would be mined as if it were a survey.

Cited titles are resolved against OpenAlex with a fuzzy-similarity floor of 88 on the title, so a wrong paper is rarely attached to a cite key. Papers for which fewer than 60% of the cited works have a retrievable abstract are dropped from the benchmark, because the task is only well posed if the system can see most of what the human author saw.

3.1.2 Citation-relation classification (component A)

The taxonomy has seven labels: background, uses-method, uses-data, extends, compares-against, supports and contrasts. The cut that matters is between compares-against and contrasts: a baseline comparison is not a disagreement, and only the latter is evidence that two papers conflict. Because labels overlap in practice, an ordered set of tie-break rules decides: disagreement outranks agreement, which outranks extension, which outranks comparison, which outranks data use, which outranks method use, with background as the default. The definitions and the rules live in one module so the LLM prompt, the human labelling guide and the training code cannot diverge.

SciCite [24] and ACL-ARC [23] are mapped onto this taxonomy as additional training signal. The mapping is lossy and is documented as such: SciCite's method class does not separate adopting a method from extending it, and its result class covers comparison, agreement and disagreement alike; ACL-ARC's Uses does not separate method from data

and its CompareOrContrast merges neutral comparison with explicit disagreement, while its Future class has no counterpart and is dropped. Mapped rows are therefore never used as evaluation gold.

The classifier is SciBERT fine-tuned for sequence-pair classification. The first segment is the citation context with the citation under question rewritten to [[CITED]] and every other citation in the sentence rewritten to [OTHER], which is what makes the label well defined for a sentence that cites several papers at once; the second segment is the cited paper's title. The dev split is taken by citing paper rather than by row, because sentences from one paper are highly correlated and a random split would leak. Class weights are inverse-frequency,

$$w_c = \frac{N}{K n_c} \qquad (3.1)$$

where N is the number of training rows, K the number of classes and n_c the count of class c, so that the rare relations are learnable at all. Performance is reported as macro-F1 over the classes present in the gold set,

$$\mathrm{F1}_{\mathrm{macro}} = \frac{1}{|C|} \sum_{c \in C} \frac{2 P_c R_c}{P_c + R_c} \qquad (3.2)$$

with P_c and R_c the precision and recall of class c, because accuracy on this distribution is dominated by background and would hide exactly the classes the downstream stages need. Agreement between the LLM labeller and the human gold set is reported as Cohen's κ,

$$\kappa = \frac{p_o - p_e}{1 - p_e} \qquad (3.3)$$

where p_o is the observed agreement and p_e the agreement expected from the two label distributions independently, so that a labeller that simply predicts the majority class scores near zero rather than near its accuracy.

3.1.3 Extraction and verification

The Extractor turns a paper into a PaperRecord: task, methods, datasets, metrics with values and settings, limitations, and three to six claims. Each claim carries an evidence object with the section it came from and a passage copied verbatim from the input. The input is the abstract plus whatever introduction, results, conclusion and limitations text the

LaTeX source yielded, under a fixed character budget, and that exact input string is stored with the record, because the quote check is defined against it and not against the paper in the abstract.

The Verifier applies two independent checks. The first asks whether the quoted passage really occurs in the input, scored by the fuzzy partial ratio of the normalised passage against the normalised source rather than by exact matching, because LaTeX-to-text conversion leaves whitespace and ligature noise that a faithful quote still trips over. The second asks whether the claim is entailed by the passage, using a DeBERTa-v3 [31] model trained on MNLI, FEVER and ANLI, applied at sentence granularity as SummaC [5] recommends. A claim can quote correctly and still overstate what the passage says, and the second check is what catches that.

A verdict is supported when the quote score clears its threshold and the entailment probability clears the upper threshold, partially when the entailment probability falls between the two thresholds, and unsupported otherwise. The three thresholds are not chosen by hand: they are fitted by grid search on a hand-checked dev set of claim/evidence pairs, maximising macro-F1 over the three verdicts rather than accuracy, so a threshold pair cannot be selected for doing well on the majority verdict alone. Section 4.3 reports the fitted values and how they compare with the defaults.

At review time the same model scores each generated sentence against the evidence of the claims whose identifiers the Writer attached to it. The faithfulness score is the fraction of sentences the Verifier calls supported,

$$\mathrm{Faith} = \frac{1}{|S|} \sum_{s \in S} \mathbb{1}\left[\max_{e \in E(s)} P_{\mathrm{ent}}(s \mid e) \geq \tau_{\mathrm{sup}}\right] \qquad (3.4)$$

where E(s) is the set of evidence passages behind sentence s and τ_sup is the fitted upper threshold. A sentence that cites nothing has an empty evidence set and is unsupported by construction, which is the intended behaviour for a review that claims to be grounded. Sentences the Verifier rejects go back to the Writer once; if the revision does not improve the score the draft is kept and the offending sentences are flagged in the output rather than silently removed.

3.1.4 Consensus, contradictions and gaps

Dataset, metric, method and task names are normalised in three stages, cheapest first: a manual alias table checked into the repository, then fuzzy matching against the canonical names, then sentence embeddings for paraphrases that fuzzy matching misses. Anything

still unmatched keeps a tidied version of its surface form and is reported, which is how the alias file gets extended rather than how information gets lost.

Records are then grouped by (task, dataset, metric). Within a group, a relative spread wider than 15% of the group median is flagged as a value disagreement, and two papers that order the same pair of methods oppositely on the same dataset and metric are flagged as a ranking disagreement. Independently, a contrasts edge predicted by the classifier is flagged as a disagreement the authors themselves asserted, which numbers alone cannot reveal. Each flagged contradiction goes to the Adjudicator, which must classify it as a different setting, a different split or metric definition, a genuine disagreement, or insufficient evidence, using only the quoted evidence passages.

Gap detection builds a method-family × dataset matrix of paper counts. An empty or near-empty cell whose row and column are both otherwise active is a candidate gap, scored by how far the count falls below the count expected if row and column activity were independent. Two method families that both appear on a dataset but between whose papers no compares-against edge exists form an untested-comparison candidate. Because an empty cell is often empty for a good reason, the Gap Analyst is required to argue both for and against each candidate before giving a verdict, and both arguments are kept in the output.

3.1.5 Orchestration

The pipeline is a CrewAI Flow of ten steps, shown in Figure 3. Roles are narrow on purpose: the Librarian has search, snowballing and reference-verification tools and nothing else; the Extractor has no tools at all because its input is handed to it; only the Verifier may call the entailment tool; and the Writer can neither search nor verify. The approval checkpoint after corpus collection is a real gate, not a formality: the Flow stops and the corpus table is shown before any LLM call is spent on extraction.

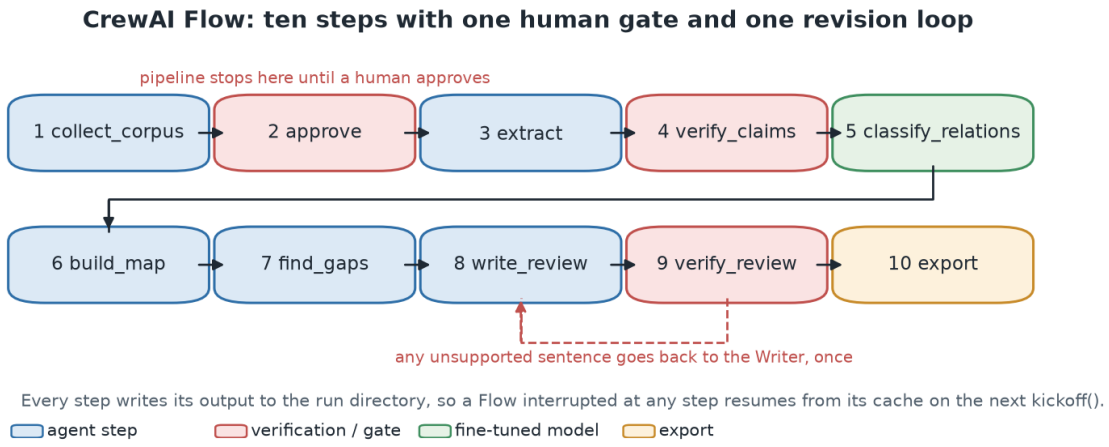


Figure 3: The ten Flow steps, the human approval gate and the single revision loop. Every step caches its output, so an interrupted run resumes.

Every step writes its output to the run directory before the next begins, which is what lets a run that dies against a rate limit continue rather than restart. The same property is what makes the Colab notebooks usable at all under a 12-hour session limit.

3.1.6 Distillation (component B)

The distillation set is built by running the hosted Extractor over training-split papers and keeping only those papers whose every claim passed the Verifier. The Verifier is therefore the filter on the student's training data, which is the point of the stage: the student cannot learn to fabricate evidence that the teacher happened to get away with. The student is a small instruction-tuned model [40] adapted with QLoRA [39] — a 4-bit NF4 base with low-rank adapters on the attention and MLP projections — which is what makes the fine-tune fit on a free T4. The comparison against the hosted extractor is on held-out test-split papers and covers JSON validity, the Verifier's own verdicts on the resulting claims, field-level agreement, latency and cost.

3.1.7 Evaluation design

The benchmark task is: given only the papers a real related-work section cites, regenerate that section. Four systems and one ablation are compared under the same model and settings. S1 is a single well-prompted call over the same cited abstracts. S2 is the crew without the Verifier. S3 is the full crew. S4 is the full crew with the distilled extractor. The ablation is S3 with the relation classifier switched off, which isolates the contribution of component A.

Overlap metrics (ROUGE-1/2/L and BERTScore against the human section) are reported beside three grounding metrics, because overlap says nothing about truth. Citation coverage is the fraction of the papers the human section cited that the generated section discusses, counted either by an inline identifier or by a distinctive overlap with the paper's title, so that a system which paraphrases instead of using identifiers is not scored at zero unfairly. Citation correctness is the fraction of asserted (sentence, cited paper) pairs for which that paper's evidence actually entails the sentence; it is the metric that catches a real identifier attached to a sentence the paper never supported. Faithfulness is as in equation (3.4).

Every mean is reported with a percentile bootstrap 95% confidence interval,

$$\mathrm{CI}_{95} = \left[\hat{\theta}^{*}_{(0.025B)},\ \hat{\theta}^{*}_{(0.975B)}\right], \quad B = 2000 \qquad (3.5)$$

where the resampled statistics are the means of B bootstrap resamples of the per-example scores. At pilot sample sizes these intervals are wide, and reporting them is what prevents a difference of a few ROUGE points from being read as a result.

Leakage is controlled by the submission window. Training papers come from 2023-2024 and test papers from a window that begins after the knowledge cutoff of every model evaluated. The Verifier's thresholds, the classifier's dev split and the human gold set all come from the training split, so nothing is tuned on the benchmark. The dates and the residual risks — chiefly that the cited papers are necessarily old and so plausibly inside every model's training data, even though the target text is not — are recorded in docs/leakage.md and in Section 4.1.

3.2 System requirements

The project is built to run in two places: a free Colab T4 session for the heavy stages, and an Apple Silicon laptop with Metal acceleration for development and the pilot runs reported in Chapter 4. Table 2 lists what was actually used; the versions are those recorded at run time, not the minimum the code would tolerate.

Table 2: System requirements and the exact versions used for the runs reported in Chapter 4.

Component	**Local (pilot runs)**	**Google Colab (full-scale runs)**
Hardware	Apple Silicon, arm64, Metal (MPS)	NVIDIA T4, 16 GB, free tier (~12 h session limit)
Operating system	Darwin 25.5.0	Ubuntu (Colab runtime)
Python	3.12.15	3.11+ (Colab default)
PyTorch	2.14.1 (MPS backend)	same wheel, CUDA backend
transformers	5.20.2	installed by the notebook
CrewAI	1.15.28	installed by the notebook
LiteLLM	not available	installed by the notebook
scikit-learn	1.9.1	installed by the notebook
sentence-transformers	6.1.0	installed by the notebook
peft	0.21.2	plus bitsandbytes (CUDA only) for 4-bit QLoRA
Relation encoder	allenai/scibert_scivocab_uncased	same checkpoint
Entailment model	MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli	same checkpoint
Embedding model	sentence-transformers/all-MiniLM-L6-v2	same checkpoint
Distillation base	not run locally (4-bit needs CUDA)	Qwen/Qwen2.5-1.5B-Instruct
LLM provider	not available	same, via Colab secrets
LLM model	not available	same model string
External APIs	arXiv, OpenAlex, Semantic Scholar	same

API access is through a free tier, which shapes the design more than the hardware does. The rate limiter is a token bucket on both requests and tokens per minute, configured per provider; every long-running step is append-only and keyed, so exhausting a daily quota means resuming tomorrow rather than starting again; and relation labelling batches several contexts into one request, which is roughly an eightfold saving on the request budget. No

API key appears anywhere in the repository: keys are read from Colab secrets when running there and from a gitignored .env file otherwise.

CHAPTER 4
ANALYSIS AND DISCUSSION

Every number in this chapter is read from a file in results/ that the code wrote, and every figure is generated from those files by scripts/make_figures.py. Where a stage was not run, that is stated rather than estimated. The scale of each run is given with its results, because pilot-scale confidence intervals are wide and a point estimate on its own would overclaim.

4.1 Dataset statistics and mining yield

The pilot listed 200 arXiv submissions in cs.CL and cs.IR (150 from the training window 202301–202412 and 50 from the test window 202606–202609). Of those, 189 had usable LaTeX source (94%); the remainder are PDF-only submissions, which arXiv permits and which no parser can recover. 189 parsed successfully and 155 had an identifiable related-work section (78% of papers listed). After requiring at least three cited works and at least 60% abstract coverage, 80 papers remain as benchmark examples and 15,704 citation contexts were written. Figure 4 shows the attrition at each step.

References are resolved by identifier first and by title only as a fallback, because an identifier resolves a reference exactly where a title match resolves it merely probably, and because the fallback is what costs money. Of the references cited from related-work sections, 1,124 carried an arXiv identifier and all 1,124 of them resolved through the arXiv API, every one with an abstract; for a corpus mined from arXiv this is also the more authoritative source, since it returns the abstract of the version actually cited. A further 738 carried a DOI, resolved fifty at a time through OpenAlex, of which 198 came back (160 with an abstract). That left 1,830 references with no identifier at all, which had to be resolved by title search; 505 matched above the similarity floor of 88 and 450 of those yielded an abstract. The title-search pass did not finish: OpenAlex's free tier is a small daily budget per key, and it ran out partway through. Every reference resolved so far is cached, so re-running after the reset continues rather than restarts, and the benchmark reported below is therefore a lower bound on what this corpus yields. The gap between a reference being identified and its abstract being retrievable is the main reason benchmark examples are dropped, since a paper can be identified without an abstract being available to redistribute.

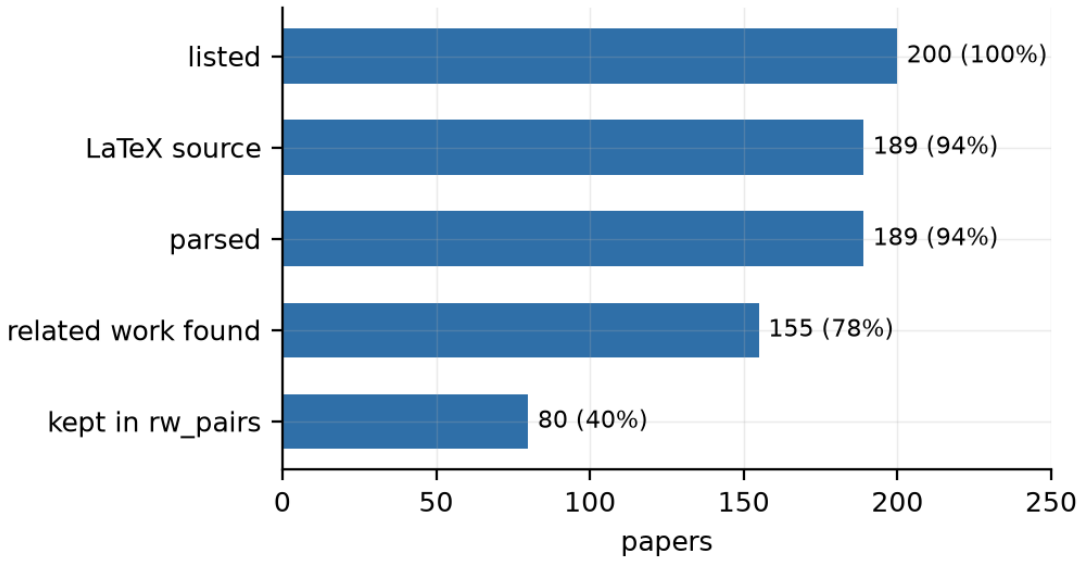


Figure 4: Mining yield: papers surviving each step, as counts and as a percentage of the submissions listed.

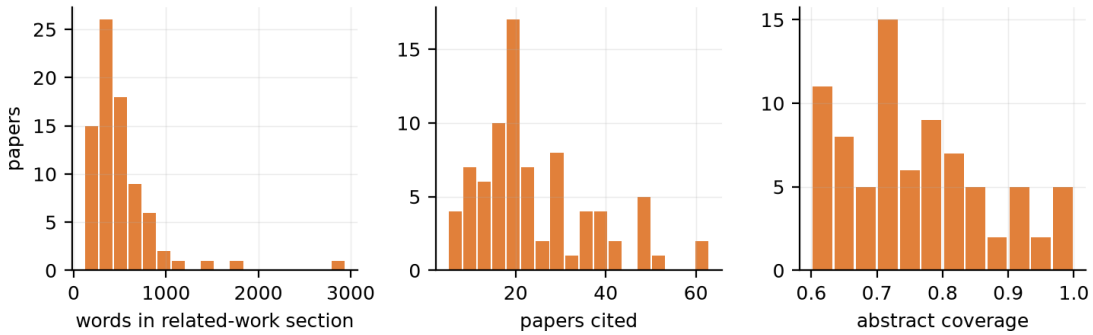


Figure 5: Distribution of related-work length, number of cited papers and abstract coverage across the mined benchmark examples.

Table 3: Stage 1 mining yield at pilot scale.

Step	Papers	% of listed
Submissions listed	200	100.0%
With LaTeX source	189	94.5%
Parsed without error	189	94.5%
With a related-work section	155	77.5%
Kept as benchmark examples	80	40.0%

Table 4: Dataset statistics for the mined arXiv benchmark and the external Multi-XScience benchmark.

Statistic	Mined arXiv split	Multi-XScience
Benchmark examples	80	300
Median related-work length (words)	420	119
Median papers cited per example	20	5
Median abstract coverage	0.74	0.857
Citation contexts	15,704	—

… of which inside a related-work section	3,652	—
Distinct citing papers	185	—

The time split is disjoint: the newest training paper is dated 2024-12-31 and the oldest test paper 2026-06-30.

The residual leakage risk is that the cited papers, being older than the citing paper by construction, are plausibly inside every evaluated model's training data. That cannot be avoided for this task and it does not give a model access to the target text, which is what is scored; it is recorded here rather than dismissed.

4.2 Citation-relation classification

Stage 2 classification is not reported here. results/stage2_benchmark.json is absent, so neither the LLM agreement nor the fine-tuned classifier can be reported. Re-run it with python scripts/run_stage2.py label, then sample_gold.py, then run_stage2.py train and bench, then rebuild this report; the text and the table will appear automatically.

4.3 Verifier calibration and claim verdicts

Verifier calibration is not reported here. results/verifier_calibration.json is absent, so the thresholds in use are the hand-chosen defaults rather than fitted values, and the claim verdicts below should be read as uncalibrated. Re-run it with python scripts/run_stage3.py make-dev, label data/verifier_dev.jsonl, then run_stage3.py calibrate, then rebuild this report; the text and the table will appear automatically.

Claim verification is not reported here. no results/stage3_verify_*.json file exists. Re-run it with python scripts/run_stage3.py extract and then verify, then rebuild this report; the text and the table will appear automatically.

4.4 Consensus map, contradictions and gaps

The end-to-end Flow run is not reported here. results/flow_summary.json is absent. Re-run it with python scripts/run_flow.py --topic "..." --n-papers 15, then rebuild this report; the text and the table will appear automatically.

4.5 System comparison

The S1–S4 comparison is not reported here. results/eval_arxiv.json is absent. Re-run it with python scripts/run_eval.py --systems S1 S2 S3 --limit 30, then rebuild this report; the text and the table will appear automatically.

4.6 Distilled extractor

Stage 6 distillation is not reported here. results/stage6_distill.json is absent. 4-bit QLoRA requires bitsandbytes, which is CUDA-only, so this stage is a Colab-T4 job and was not run on the development machine. Re-run it with notebooks/05_distill_extractor_qlora.ipynb on a Colab T4, then rebuild this report; the text and the table will appear automatically.

4.7 Qualitative examples and error analysis

No cached generations are available, so no qualitative comparison or error analysis is reported here. Run scripts/run_eval.py and rebuild this report.

4.8 Limitations and threats to validity

• Pilot sample sizes. The runs reported here are pilots, sized so that the whole pipeline could be executed end to end on one laptop. The bootstrap intervals in Table 5 are correspondingly wide, and no claim of a significant difference between systems is made where those intervals overlap.

• LLM-labelled training data. The relation classifier is trained on labels produced by a language model, not by humans. Section 4.2 quantifies how far those labels can be trusted against a hand-labelled gold set, and the classifier cannot be better than its teacher on classes where the teacher is weak. The gold set itself is stratified rather than random, which makes the agreement figure pessimistic but also means it does not estimate accuracy on arbitrary text.

• Single annotator. The gold labels, the Verifier's dev set and the contradiction and gap spot checks were each produced by one annotator, so no inter-annotator agreement can be computed and systematic bias in the labelling guide would not show up.

• NLI model errors. The Verifier inherits the failure modes of its entailment model: it is weak on numerical and quantitative entailment, which is exactly what many scientific claims turn on, and it treats hedged statements inconsistently. A claim marked unsupported is therefore evidence of a problem, not proof of one.

• Quote matching is not provenance. A fuzzy match confirms that a passage resembles something in the input text. It does not confirm that the passage means in context what the claim says it means, and a short generic passage can match in several places.

• Leakage controls are partial. The target related-work text is time-split beyond every evaluated model's cutoff, but the cited papers are necessarily older and plausibly

memorised. Topic familiarity may therefore help a model even though the target text cannot.

• Benchmark construction bias. Only papers with usable LaTeX, an identifiable related-work section and at least 60% abstract coverage survive. That selects for conventionally structured papers from authors who submit source, and the resulting benchmark is not a random sample of the literature.

• ROUGE against one reference. There are many defensible related-work sections for a given set of cited papers, and only one is in the benchmark. This is why the grounding metrics are reported beside the overlap metrics rather than beneath them.

• Normalisation is incomplete. Dataset, metric and method names are merged by an alias table plus fuzzy matching; unmatched names keep their surface form, so a consensus group can be split in two and a contradiction consequently missed.

• One provider, one run. Every system used the same provider and settings, which makes the comparison fair but says nothing about how the conclusions transfer to a different model family, and each configuration was run once, so run-to-run variance is not measured.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project built a literature-review assistant in which grounding is enforced rather than requested. Claims must quote their evidence, a separate discriminative model with fitted thresholds decides whether the quote supports the claim, review sentences carry the identifiers of the claims behind them, and anything that fails the check is revised once and then flagged rather than hidden. Two components were fine-tuned on data reverse-engineered from real papers' related-work sections, and the whole pipeline was evaluated against human-written sections on a time-split benchmark.

The full system comparison was not completed when this report was built, so no conclusion is drawn about whether the multi-agent pipeline outperforms a single call. Section 4.5 states exactly which systems were run.

The secondary findings are worth stating separately. A model asked to quote verbatim usually does, and still writes claims its own quotation does not entail, which is why the two Verifier checks are independent rather than one. Using the Verifier as the filter on distillation data is a cheap way to keep a student model from inheriting its teacher's unsupported extractions. And the cost of the agent pipeline is large and measurable; reporting it beside the quality numbers is what makes the trade visible at all.

Finally, the honesty constraints shaped the result as much as the design did. Every number here is read from a file the code wrote, every figure is generated from those files, six candidate references were dropped because no source would confirm them, and the stages that could not be run on the available hardware are named rather than estimated.

5.2 Suggestions for future directions

Seven directions follow from the limitations in Section 4.8, roughly in order of how much they would change the conclusions.

1. Scale the pilot. Every number in Chapter 4 comes from a pilot sized to run on one laptop. The notebooks are written for the full configuration (roughly 3,000 training and 300 test papers, 5,000 labelled contexts); running them would narrow the intervals enough for the system comparison to be decidable rather than suggestive.

2. Replace LLM labels with human ones where they matter. The relation classifier is limited by its teacher on exactly the rare classes the contradiction map depends on. A few hundred human labels concentrated on contrasts and supports would be worth more than ten thousand more LLM labels.

3. Measure inter-annotator agreement. The gold set, the Verifier's dev set and the contradiction spot checks each had one annotator. A second annotator on a subset would turn the labelling guide from an assertion into something testable.

4. Train the Verifier rather than only calibrating it. Thresholds on an off-the-shelf NLI model are a blunt instrument, and the model is weakest on the numerical entailment that scientific claims turn on. Fine-tuning an entailment model on claim/evidence pairs from this domain is the obvious next component.

5. Extract from full text rather than from abstracts. When the Flow runs on an OpenAlex corpus it sees only abstracts, which is why the relation classifier contributes little there: there are no in-corpus citation sentences to classify. Resolving corpus papers to their arXiv sources would let the typed citation graph be built from real citation contexts.

6. Complete and then stress the distillation. Component B is a Colab-T4 job; beyond running it, the question worth asking is whether a student trained only on Verifier-approved data generalises better than one trained on everything, which is a controlled experiment this codebase can already run.

7. Attempt component C. Fine-tuning a related-work generator directly on the mined (cited abstracts → section) pairs was left as a stretch goal. It should be gated through the same Verifier, so that a model trained to imitate human sections is still held to the grounding standard the rest of the system enforces.

REFERENCES

Journal / Conference Papers

[1] Joshua Maynez et al., “On Faithfulness and Factuality in Abstractive Summarization”, Annual Meeting of the Association for Computational Linguistics, 2020, doi:10.18653/v1/2020.acl-main.173

[2] Ziwei Ji et al., “Survey of Hallucination in Natural Language Generation”, ACM Computing Surveys, 2022, doi:10.1145/3571730

[3] Alex Wang, Kyunghyun Cho and Mike Lewis, “Asking and Answering Questions to Evaluate the Factual Consistency of Summaries”, Annual Meeting of the Association for Computational Linguistics, 2020, doi:10.18653/v1/2020.acl-main.450

[4] Wojciech Kryściński et al., “Evaluating the Factual Consistency of Abstractive Text Summarization”, Conference on Empirical Methods in Natural Language Processing, 2020, doi:10.18653/v1/2020.emnlp-main.750

[5] Philippe Laban et al., “SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization”, arXiv, 2021, doi:10.48550/arxiv.2111.09525

[6] Guohao Li et al., “CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society”, neural information processing systems, 2023, doi:10.52202/075280-2264

[7] Joon-Sung Park et al., “Generative Agents: Interactive Simulacra of Human Behavior”, ACM Symposium on User Interface Software and Technology, 2023, doi:10.1145/3586183.3606763

[8] Aman Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback”, neural information processing systems, 2023, doi:10.52202/075280-2019

[9] Noah Shinn et al., “Reflexion: language agents with verbal reinforcement learning”, neural information processing systems, 2023, doi:10.52202/075280-0377

[10] Abigail See, Peter J. Liu and Christopher D. Manning, “Get To The Point: Summarization with Pointer-Generator Networks”, Annual Meeting of the Association for Computational Linguistics, 2017, doi:10.18653/v1/p17-1099

[11] Mike Lewis et al., “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension”, Annual Meeting of the Association for Computational Linguistics, 2020, doi:10.18653/v1/2020.acl-main.703

[12] Jingqing Zhang et al., “PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization”, arXiv, 2019, doi:10.48550/arxiv.1912.08777

[13] Arman Cohan et al., “A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, doi:10.18653/v1/n18-2097

[14] Shashi Narayan, Shay B. Cohen and Mirella Lapata, “Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization”, arXiv, 2018, doi:10.48550/arxiv.1808.08745

[15] Ahmed AbuRa’ed et al., “Automatic related work section generation: experiments in scientific document abstracting”, Scientometrics, 2020, doi:10.1007/s11192-020-03630-2

[16] Yongzhen Wang, Xiaozhong Liu and Zheng Gao, “Neural Related Work Summarization with a Joint Context-driven Attention Mechanism”, Conference on Empirical Methods in Natural Language Processing, 2018, doi:10.18653/v1/d18-1204

[17] Kelvin Luu et al., “Explaining Relationships Between Scientific Documents”, preprint, 2021, doi:10.18653/v1/2021.acl-long.166

[18] Yao Lu, Yue Dong and Laurent Charlin, “Multi-XScience: A Large-scale Dataset for Extreme Multi-document Summarization of Scientific Articles”, Conference on Empirical Methods in Natural Language Processing, 2020, doi:10.18653/v1/2020.emnlp-main.648

[19] Kyle Lo et al., “S2ORC: The Semantic Scholar Open Research Corpus”, Annual Meeting of the Association for Computational Linguistics, 2020, doi:10.18653/v1/2020.acl-main.447

[20] Tianyi Zhang et al., “Benchmarking Large Language Models for News Summarization”, Transactions of the Association for Computational Linguistics, 2024, doi:10.1162/tacl_a_00632

[21] Awais Athar and Simone Teufel, “Context-Enhanced Citation Sentiment Detection”, preprint, 2012

[22] Saeed-Ul Hassan et al., “Mining the Context of Citations in Scientific Publications”, Lecture notes in computer science, 2018, doi:10.1007/978-3-030-04257-8_32

[23] David Jurgens et al., “Measuring the Evolution of a Scientific Field through Citation Frames”, Transactions of the Association for Computational Linguistics, 2018, doi:10.1162/tacl_a_00028

[24] Arman Cohan et al., “Structural Scaffolds for Citation Intent Classification in Scientific Publications”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2019, doi:10.18653/v1/n19-1361

[25] Iz Beltagy, Kyle Lo and Arman Cohan, “SciBERT: A Pretrained Language Model for Scientific Text”, preprint, 2019, doi:10.18653/v1/d19-1371

[26] Alexander R. Fabbri et al., “SummEval: Re-evaluating Summarization Evaluation”, Transactions of the Association for Computational Linguistics, 2021, doi:10.1162/tacl_a_00373

[27] Tanya Goyal and Greg Durrett, “Annotating and Modeling Fine-grained Factuality in Summarization”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021, doi:10.18653/v1/2021.naacl-main.114

[28] Zhang, Tianyi et al., “BERTScore: Evaluating Text Generation with BERT”, arXiv, 2019, doi:10.48550/arxiv.1904.09675

[29] James Peter Thorne et al., “FEVER: a Large-scale Dataset for Fact Extraction and VERification”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, doi:10.18653/v1/n18-1074

[30] David Wadden et al., “Fact or Fiction: Verifying Scientific Claims”, Conference on Empirical Methods in Natural Language Processing, 2020, doi:10.18653/v1/2020.emnlp-main.609

[31] Pengcheng He et al., “DeBERTa: Decoding-enhanced BERT with Disentangled Attention”, arXiv, 2020, doi:10.48550/arxiv.2006.03654

[32] Jason Wei et al., “Chain-Of-Thought Prompting Elicits Reasoning in Large Language Models”, neural information processing systems, 2022, doi:10.52202/068431-1800

[33] Timo Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools”, neural information processing systems, 2023, doi:10.52202/075280-2997

[34] Patrick Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, UCL Discovery, 2020

[35] Vladimir Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering”, Conference on Empirical Methods in Natural Language Processing, 2020, doi:10.18653/v1/2020.emnlp-main.550

[36] Jakub Lála et al., “PaperQA: Retrieval-Augmented Generative Agent for Scientific Research”, arXiv, 2023, doi:10.48550/arxiv.2312.07559

[37] Yijia Shao et al., “Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2024, doi:10.18653/v1/2024.naacl-long.347

[38] Teo Sušnjak et al., “Automating Research Synthesis with Domain-Specific Large Language Model Fine-Tuning”, ACM Transactions on Knowledge Discovery from Data, 2025, doi:10.1145/3715964

[39] Tim Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs”, neural information processing systems, 2023, doi:10.52202/075280-0441

[40] Qwen et al., “Qwen2.5 Technical Report”, arXiv, 2024, doi:10.48550/arxiv.2412.15115

Reference / Hand Books

No textbook reference is cited. The two candidates considered (Jurafsky and Martin's Speech and Language Processing, and Manning, Raghavan and Schütze's Introduction to Information Retrieval) could not be confirmed as works by OpenAlex, Semantic Scholar or the arXiv API — a title search returns reviews of them rather than the books themselves — and this report does not cite what it cannot verify. docs/references_candidates.txt records the decision.

Web

[1] CrewAI documentation, https://docs.crewai.com

[2] LiteLLM documentation, https://docs.litellm.ai

[3] OpenAlex API documentation, https://docs.openalex.org

[4] arXiv API user manual, https://info.arxiv.org/help/api/

[5] Hugging Face model hub, https://huggingface.co/models

[6] Multi-XScience dataset repository, https://github.com/yaolu/Multi-XScience